Recalibrated roadmap · academic year 2026–27

Recalibrated plan - academic year 2026-27 - five-person team - WTO endgame unchanged

This is NOT the project mandate. The mandate is zbridge-asm-roadmap.pdf at the repository root, which is never edited. This document supersedes only that PDF's timeline (p.3 phase windows, p.7 solo time budget), authorised by ADR 0005. Phase definitions, deliverables and the WTO endgame are unchanged.

Authorised by ADR 0005. Supersedes the phase-window column of `zbridge-asm-roadmap.pdf` p.3 and the solo time budget on p.7. **Supersedes no phase definition and no deliverable.**

- **Prepared:** 2026-07-30
- **Endgame, unchanged:** `WTO(message string) error` in pure Go (Plan 9) assembly for s390x, issuing `SVC 35` on z/OS. No cgo. No Language Environment dependency beyond the `Malloc31` touchpoint.
- **Team:** five, lead + four contributors. Mentor: Jürgen Holtz (IBM).
- **Read with:** `roadmap-errata.md` — four roadmap statements are superseded, one of which produces a broken parameter list if implemented as written.

Part I · Where the project actually is

Every status cell below points at a file. Where no evidence exists, the cell says so rather than estimating.

1. The spine: the three unknowns

The project's real progress measure is not phases — it is which of the three unknowns in ADR 0001 §3 have been retired.

	Unknown	Status	Retired by
U1	Does our Go assembly emit correct s390x, and does the Go ABI / frame contract hold on a big-endian 64-bit target?	● **RETIRED** 2026-07-25	`E-L-s390x-port-qemu-2026-07-25.md`
U2	Is the WTO parameter list byte-correct; does `SVC 35` accept it?	● **RETIRED** 2026-07-26	`E1-E3-wto-layout-and-svc35-2026-07-26.md`
U3	`GOOS=zos` toolchain, `Malloc31` / below-the-bar, AMODE 31↔64, extended WPL, USS	● **OPEN — nothing emulates this**	Real, entitled z/OS only

Two of three fell without a mainframe. That is the single most important fact about the project's position, and it is why the licensing answer being "no" cost the schedule nothing.

U3 remains one unknown — it is not re-split by ADR 0005 §6. But it contains an ordering fact that drives the whole forward plan:

> *There is no `zos/s390x` in `go tool dist list` (go1.26.3, verified against the toolchain source). **IBM's Go fork is a hard dependency of every z/OS-side step**, and it is a different entitlement from access to a z/OS system. You cannot run a binary on a machine you have no compiler for — so the toolchain is upstream of the access question in execution order, even though both are unresolved.*

2. Phase-by-phase, against the roadmap's own definitions

Phase	Roadmap definition	Status 2026-07-30	Evidence / note
0 Foundation	Assembler theory + Go asm reading; absorbed into 1+2	**Absorbed as planned.** One deliverable outstanding: the 2–3 page s390x cheat sheet, which p.3 relocates to the end of Phase 2	roadmap p.3
1 Go assembly on x86	Six exercises, toolchain checkpoint + 5	● **COMPLETE** 2026-07-05	`2026-07-05-phase1-…`
1b Port to s390x	Five `_s390x.s` bodies + benchmark table	● **COMPLETE.** 29 tests pass on `linux/s390x` under QEMU 10.2.1. Benchmark deliverable **relocated to 3c**, not dropped	`E-L-…`; ADR 0004 §4
2 go-recordio `utils.s` annotation	Four deliverables, "the most important pre-z/OS phase"	● **NOT STARTED.** Its window (mid-July → early August) is closing. **This is the critical path**	roadmap p.5
3a Toolchain validation	`GOOS=zos GOARCH=s390x`, run on USS	● **BLOCKED** — upstream Go has no `zos/s390x` target	goal-prompt §2
3b WTO scaffold, six steps	The endgame implementation	● **4 of 6 done, plus half a fifth** — off-mainframe. See §3	`E1-E3-…`
3c Validation & documentation	Console confirmation, latency baseline, public module	● **NOT STARTED** — needs z/OS	roadmap p.6
4 Stretch: WTOR, Name/Token, EBCDIC package	Only after WTO ships	● **One item shipped early** — `zbridge/codepage` is the EBCDIC package in all but publication	roadmap p.7

3. Phase 3b, step by step — the endgame's own checklist

This table is the most useful single view of the project, because Phase 3b *is* the thesis.

#	Step	Status	How, and what still stands in the way
1	Allocate a parameter buffer below the bar via `Malloc31`	● **Open**	U3. No emulator reaches it. Gated on Phase 2 for the *pattern* and on z/OS for the *execution*
2	Translate UTF-8 → EBCDIC IBM-1047 via `AtoE`	● **Done**	The EBCDIC in the parameter list a real `SVC 35` accepted is `codepage.AtoE` output
3	Construct the WTO parameter list	● **Done**	Layout read from IBM's own macro expansion; Go's construction accepted. **The roadmap's own description of this step is wrong** — errata E1
4	Load R1 with the parameter-list address	● **Half done**	MVS linkage verified twice (E2, E3). The AMODE 31↔64 context around it is U3
5	Issue `SVC 35`	● **Done**	Issued raw, no macro, twice. On z/OS it must be hand-encoded `BYTE $0x0A; BYTE $0x23` — Go's s390x assembler has no `SVC` mnemonic
6	Read R15, map to a Go error	● **Not retirable off-mainframe**	**System-dependent.** MVS 3.8j issues no return code for a single-line WTO; z/OS does. Errata E2 / ADR 0004 §2.2

What this converts. Rung T3 on borrowed machine time changes from *"invent a parameter list while an operator watches"* into *"port a verified one."* That is the entire return on the emulation programme.

4. What exists in the repository that the roadmap does not contain

Not deviations — structures built underneath the roadmap to make it survivable.

Structure	What it is	Where
U1/U2/U3 decomposition	Splits "we need a mainframe" into three questions with different oracles	ADR 0001 §3
The E-ladder, E0→E3	Four off-mainframe rungs that pre-validated most of Phase 3b. **All four passed 2026-07-26**	ADR 0001 §6
`zbridge/` **production module**	Seven packages plus a command, BSD-3-Clause. Governing rule: *every exported function whose behaviour depends on an unretired unknown returns a typed error naming that unknown, on every platform, until its rung passes*	ADR 0003
Pre-registered hypotheses	H001 (MVS as z/OS oracle) **open**; H002 (port equivalence) **resolved** on C1/C2/C4, C3 untested	`docs/hypotheses/`
The evidence convention	Every result carries machine, guest OS, architecture, emulator version, and which unknown it speaks to	ADR 0001 §7
`mvsjob.sh`	The headless TK5 harness — `up` / `run` / `cmd` / `down`	`docs/runbooks/`
Roadmap errata	Four superseded roadmap statements, each with its ADR and evidence	`docs/roadmap-errata.md`

5. Verified today, not recalled

Run 2026-07-30 on `windows/amd64`, Go 1.26.3:

Gate	Result
`go vet ./...` on `zbridge`	clean
`go test ./...` on `zbridge`	**27 pass, 0 fail**
`go test ./...` across the six lab modules	**19 pass, 0 fail** (`syscall-linux` contributes 0 on Windows — it is Linux-constrained by design)
`GOOS=linux GOARCH=s390x go build ./...`	**clean on all six** — `ebcdic`, `strmanip`, `regs`, `bytecmp`, `syscall-linux`, `zbridge` (`add/` is excluded and is not supposed to cross-build)

The 29-test `linux/s390x` figure is from the QEMU evidence file and was **not** re-run today.

6. The honest score

- **Complete:** Phases 1 and 1b; the entire E-ladder; the production module scaffold.
- **Substantially complete:** Phase 3b — four of six steps plus half a fifth, ahead of z/OS.
- **Not started, and on the critical path:** Phase 2.
- **Blocked on entitlements that do not exist yet:** Phases 3a, 3c, and Phase 3b steps 1 and 6.
- **Unresolved risk, unchanged since the roadmap was written:** z/OS access — still *"the single biggest risk"* (p.7), and now joined by the Go fork, which the roadmap did not anticipate because it assumed the compiler existed.

Part II · What remains, phase by phase

Each phase below states its **gate** — the specific observable that ends it. A phase is not done because its window closed.

Phase 2 · go-recordio deep annotation — do this first

Owner: W1, reviewed by W4 · **Needs:** nothing. No hardware, no access, no entitlement. **Window:** now → late September 2026.

The roadmap's most important pre-z/OS phase, and the only route to the two Phase 3b steps still open. `Malloc31` and the SAM31/SAM64 switching are exactly what that file contains.

Before starting, read errata E4. The roadmap's target path does not exist: there is no `utils/utils.s`. The file is `v2/utils/utils.s` — 154 lines, 6,934 bytes — and `Malloc31` is in `v2/utils/utils.go` at line 47, in Go rather than assembly.

The nine routines, which is how this phase parallelises across the team:

`IefssreqX` · `Bpxcall` · `Svc8` · `Svc9` · `Call24` · `Call31` · `Call64` · `Deref` · `Pc31`

Deliverables — all four from roadmap p.5, unchanged by ADR 0004 §5, which ruled that Phase 2 runs whole rather than narrowed to the two primitives that unblock Phase 3b:

1. Instruction-by-instruction annotated walkthrough, in a format fit for publication.
2. CVT → JESCT → SSREQ navigation diagram with byte offsets called out.
3. Documentation of the SAM31/SAM64 switching pattern around the `BALR`.
4. Identification of which patterns are SSREQ-specific and which transfer directly to WTO.

Plus the deliverable Phase 0 relocated here: **the 2–3 page s390x assembly cheat sheet**.

Gate: all five artifacts committed, and the walkthrough sent to the mentor for the review roadmap p.5 already asked for.

Two findings this phase can bank immediately, already confirmed from the file listing: `SAM31`/`SAM64` are hand-encoded (`BYTE $0x01; BYTE $0x0D` / `$0x0E`), and `SVC 8`/`SVC 9` are hand-encoded as `BYTE $0x0A; …`. **IBM's own module uses exactly the technique this project derived independently** — which is the strongest available answer to the open question of whether hand-encoding `SVC 35` is idiomatic or merely expedient.

Phase 3a · Toolchain validation — blocked, and it is the critical path

Owner: W2 · **Needs:** IBM's Go fork **and** a z/OS system. **Window:** fall 2026, if the dependency lands.

Cross-compile the Phase 1b exercises `GOOS=zos GOARCH=s390x` and run them on z/OS USS. This validates the assumption Phase 1b rests on: that what works on s390x Linux works on z/OS unmodified.

Why it is the critical path and not merely blocked: upstream Go cannot target z/OS at all. `"zos"` is in `internal/syslist` so `_zos.go` build constraints parse, but the target is absent from `internal/platform`. Every z/OS-side step in the project is downstream of obtaining and building that fork.

Work that can start before the fork arrives — and should, because it costs nothing to be ready:

- Write the `_zos.go` / `_zos_s390x.s` files against the constraint names that already parse.
- Keep the `zbridge` stub/`zos` split building on both platforms (it already does).
- Prepare the T-ladder Day-0 checklist so the first hour of access climbs a rung.

Gate: one Phase 1 exercise binary, built by the fork, runs on z/OS USS and prints a correct result.

Phase 3b · The WTO scaffold — two steps left

Owner: W1 + W2 · **Needs:** Phase 2 for step 1's pattern; z/OS for both steps' execution.

Step	What remains
1 `Malloc31` / below-the-bar	The pattern comes out of Phase 2. The execution needs z/OS. This is the one Language Environment touchpoint the thesis permits
4 AMODE context	The linkage half is done. The AMODE 31↔64 switch around the call has no MVS 3.8j analogue and must be validated on z/OS

Step	What remains
6 Return code	**Re-scoped, not merely blocked.** The roadmap's wording is wrong for MVS and unverified for z/OS. Corrected wording: *"read the service's result — a return code in R15 on z/OS, nothing on MVS 3.8j — and translate to a Go error where one exists."* `zbridge.Error` already carries `HasCode bool` for exactly this

Gate: `WTO("...")` called from a Go program on z/OS puts the message on the operator console, and the function returns an error value that correctly reflects what the service reported.

Phase 3c · Validation, measurement, publication

Owner: W4 + W2 · **Needs:** a working Phase 3b. **Window:** spring 2027.

- Confirm the message on the operator console; document every step.
- **Latency comparison against `__console2()` through cgo** — a *measurement baseline only*, never a shipped path (ADR 0004 §3). The claim being tested is *"as fast as the C path, with no C, no LE, and a static binary."*
- **The relocated Phase 1b benchmark table**: amd64 lookup loop vs s390x `TR`, on real hardware. It cannot be produced under emulation — QEMU and Hercules implement `TR` as a software loop, so timing it measures the emulator. This was pre-registered as an explicit non-claim in H002 *before* any code ran.
- **Deliverable:** a public Go module, BSD-3-Clause to match go-recordio, with implementation, tests, and a findings document covering what worked, what did not, and what needed workarounds.

Gate: the module is published and the findings document is complete. Publication is autonomy boundary 3 — the owner's call, not the team's.

Phase 4 · Stretch, only after WTO ships

Owner: W1 + W4 · **Window:** spring 2027.

1. **WTOR** — Write To Operator with Reply. The architecturally interesting one: the Go wrapper has to bridge the legacy asynchronous ECB event model to goroutines and channels instead of blocking the OS thread on a `WAIT` macro. `console.WTOR` already has its signature and returns a typed error.
2. **Name/Token services** — `IEAN4RT` / `IEANTCR`. Standard MVS CALL linkage rather than direct SVC, same parameter-list discipline.
3. **Go-native EBCDIC package** — largely already built as `zbridge/codepage`. What remains is polish, extraction, and publication. Lowest novelty, highest immediate community utility.

Cross-cutting track · Access and entitlement

Owner: W5 (lead) · **Runs continuously from now.**

This is not a phase; it is a standing pursuit, and it is the only work item whose failure the plan cannot absorb.

Question	Status	Why it is separate
A compiler — obtain and build IBM's Go fork	Open. Never asked, because the meeting slipped	A compiler that emits z/OS binaries is a different entitlement from a machine to run them on
A system — entitled z/OS access	Open. No entitlement exists; ADR 0002 was withdrawn on exactly this	ZD&T, Wazi aaS, a shared LPAR, or the more open Z environment — the option decides the timeline

Merging these two questions is the specific mistake to avoid. Asking only about access gets an answer that still leaves the project unable to build anything.

Part III · The schedule

Three blocks, anchored to the 2026–27 academic year. **No unentitled work is ever scheduled behind entitled work** — the same principle the E-ladder was built on, applied to the calendar.

Block A · Pre-semester · now → late September 2026

Workstream	Deliverable	Gate
W1	Phase 2, all four deliverables + cheat sheet	Walkthrough sent to the mentor
W2	`_zos` files written against parsing constraints; fork acquisition pursued	Both platforms still build
W3	`mvsjob.sh` productised; CI runs vet + test + s390x cross-build on every PR	A PR that breaks the cross-build fails automatically
W4	Provenance audit of everything committed; the three untracked docs landed	`evidence-provenance-auditor` clean
W5	Mentor checkpoint delivered; both access questions asked	A reply on the Go fork

Block A is the block with no external dependencies. Everything in it can be finished without IBM answering anything, and it should be finished before the semester's other demands arrive.

Block B · Fall semester · October 2026 → January 2027

Priority	Work	Depends on
1	Phase 3a gate	The Go fork
2	Phase 3b steps 1 and 4	Phase 2 + z/OS
3	T-ladder T0 → T2	z/OS access
4	H002 claim C3 (Hercules-hosted Linux s390x vs QEMU)	Nothing — fills schedule gaps
5	H001 resolution: does z/OS accept the MVS-derived layout?	z/OS

The contingency is explicit. If neither entitlement lands by end of Block B, Block C runs its Plan B below rather than idling.

Block C · Spring semester · February → June 2027

Plan A — access landed: Phase 3c in full, Phase 4 items 1–3, module publication, thesis write-up against a working `WTO`.

Plan B — no z/OS by February 2027. The thesis still has a defensible contribution, and it is worth naming now rather than discovering in a panic:

1. **The Phase 2 annotated walkthrough**, published standalone. Nobody has published it; the roadmap says so and Jürgen was already asked to review it (p.5).
2. **The WPL layout and its encoder**, with the byte-for-byte oracle test against IBM's own macro output. That result is real, evidenced, and independent of z/OS.
3. **The E-ladder as methodology** — retiring two of three unknowns for a mainframe service without a mainframe is a transferable result about how to do this kind of work.
4. **The negative result on entitlement pathways**, documented with the ADR-withdrawal record. This project already treats negative results as publishable; goal-prompt §4.3 says so.

5. `zbridge` **published with the z/OS paths returning typed errors naming U3** — a library that is honest about what it has not proven is a contribution, not a failure.

Part IV · Risks

Risk	Severity	Mitigation, and its limit
No IBM Go fork	**Critical**	Cannot be mitigated technically. ADR 0004 §3 removed the cgo hedge, and the reasoning holds: the cgo route needs the fork too. This is a relationship problem, not an engineering one
No z/OS access	**Critical**	Block C Plan B. Every phase that does not need it is scheduled ahead of it
H001 falsifies — z/OS rejects the MVS-derived layout	Medium	The divergence would itself be publishable. `LayoutVerified` is a single constant and the module is built to be told it was wrong
Phase 2 slips again	Medium	It has slipped once. Block A exists to close it while nothing competes
Semester load	Medium	Five people, and Block A front-loads the unblocked work into the summer
Team doctrine drift	Medium	ADR 0005 §4 moves doctrine to the merge gate; the charter's PR template is where it bites
Mentor meetings slip	Low–Medium	Already happened once (2026-07-27). Written checkpoints go out whether or not a meeting is scheduled

What would change this document

It is a plan, not a decision. It is rewritten — not amended — when any of these happen:

1. Either entitlement resolves. The whole schedule re-cuts around machine time.
2. The team's size changes. ADR 0005 §9.1 says what folds into what.
3. A phase gate fails in a way that changes what a phase means. That becomes an ADR first.
4. The mentor rules differently on scope. His position governs; roadmap p.8 asked for exactly that and the question is still open.

The roadmap PDF is never edited. Contradictions go to `roadmap-errata.md` and are superseded by an ADR.